

Mastering Unsupervised Learning: Clustering Techniques

1. Data Preparation

We start by loading our customer data and cleaning the column names for easier access.

```
In [1]: # !pip install seaborn scipy scikit-learn
```

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram

# Load dataset
df = pd.read_csv('../data/customers.csv')
df.columns = ['customer_id', 'gender', 'age', 'income', 'spending_score']
df=df[['age', 'spending_score']]
df.head()
```

```
Out[2]:
```

	age	spending_score
0	19	39
1	21	81
2	20	6
3	23	77
4	31	40

	age	spending_score
0	19	39
1	21	81
2	20	6
3	23	77
4	31	40

```
In [4]: X=df.values
my_kmeans = KMeans(n_clusters=3)
my_kmeans.fit(X)
```

```
Out[4]:
```

▼ KMeans ⓘ ?

► Parameters

2. Finding the Optimal K:

2.1 The Elbow Method

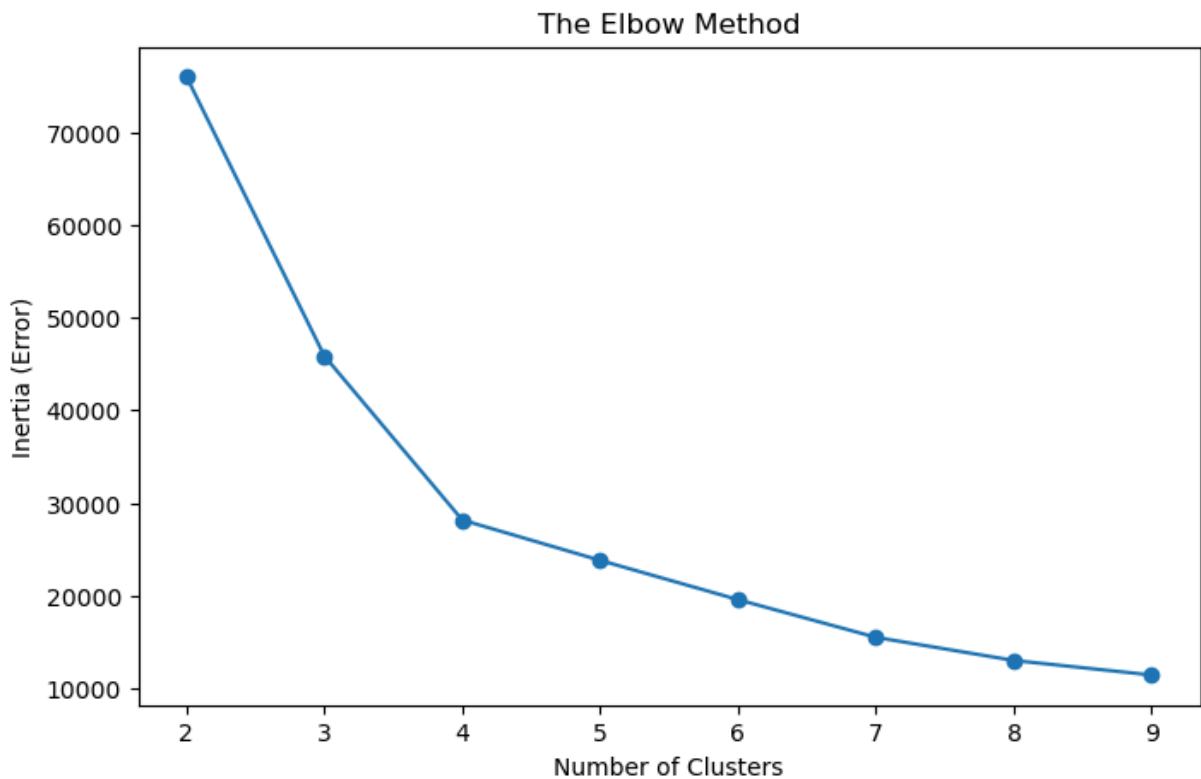
The **Elbow Method** looks at the total **Inertia** (sum of squared errors). We look for the "elbow" point where adding another cluster doesn't significantly reduce the error.

```
In [5]: errors = []
X = df[['age', 'spending_score']].values

for n in range(2, 10):
    my_kmeans = KMeans(n_clusters=n, random_state=0, n_init=10)
    my_kmeans.fit(X)
    errors.append(my_kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(2, 10), errors, marker='o')
plt.title('The Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia (Error)')
plt.show()

print("Based on the plot, the 'Elbow' appears at K=4.")
```



Based on the plot, the 'Elbow' appears at K=4.

```
In [6]: from kneed import KneeLocator
K=range(2, 10)
kneedle = KneeLocator(K, errors, curve='convex', direction='decreasing')
optimal_k = kneedle.knee
print("Optimal number of clusters:", optimal_k)
```

Optimal number of clusters: 4

2.2 Validating with Silhouette Score

The **Silhouette Score** measures how similar an object is to its own cluster compared to others.

- **Close to 1:** Well-separated clusters.
- **Close to 0:** Overlapping clusters.
- **Negative:** Likely assigned to the wrong cluster.

```
In [7]: s_scores = []
for k in range(2, 10):
    kmeans = KMeans(n_clusters=k, random_state=0, n_init=10).fit(X)
    score = silhouette_score(X, kmeans.labels_)
    s_scores.append(score)

optimal_k = np.argmax(s_scores) + 2
print(f"Optimal number of clusters based on Silhouette Score: {optimal_k}")
```

Optimal number of clusters based on Silhouette Score: 4

2.3 Stability-based method

- Answer a very practical question: "If I slightly change my data, do I get the same clusters?"
 - If yes → your clustering (and k) is reliable
 - If no → your clusters are probably arbitrary
- **Core idea:**
 1. Perturb the dataset (resample / add noise)
 2. Re-run clustering
 3. Compare cluster assignments across runs
 4. **Good k = high consistency (stability)**
- Pick the k with highest stability score

```
In [8]: from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

def stability_score(X, k, runs=10):
    # Run clustering on multiple samples using same k
    labels_list = []
    for _ in range(runs):
        idx = np.random.choice(len(X), int(0.8 * len(X)), replace=False)
        X_sample = X[idx]

        labels = KMeans(n_clusters=k, n_init=10).fit_predict(X_sample)
        labels_list.append((idx, labels))
```

```

# Find the scores of each run then calculate the stability
scores = []
for i in range(len(labels_list)):
    for j in range(i+1, len(labels_list)):
        idx1, lab1 = labels_list[i]
        idx2, lab2 = labels_list[j]

        common = set(idx1).intersection(idx2)

        if len(common) > 0:
            lab1_map = {idx1[t]: lab1[t] for t in range(len(idx1))}
            lab2_map = {idx2[t]: lab2[t] for t in range(len(idx2))}

            l1 = [lab1_map[c] for c in common]
            l2 = [lab2_map[c] for c in common]

            scores.append(adjusted_rand_score(l1, l2)) # Do these two cl

return np.mean(scores)

```

```

In [13]: # Run Stability Across k

X_values = df[['age', 'spending_score']].values
k_values = range(2, 10)
stability_scores = []

for k in k_values:
    score = stability_score(X_values, k)
    stability_scores.append(score)
    print(f"k={k}, stability={score:.4f}")

```

```

k=2, stability=0.9244
k=3, stability=0.9696
k=4, stability=0.9783
k=5, stability=0.7098
k=6, stability=0.6922
k=7, stability=0.7658
k=8, stability=0.8265
k=9, stability=0.8019

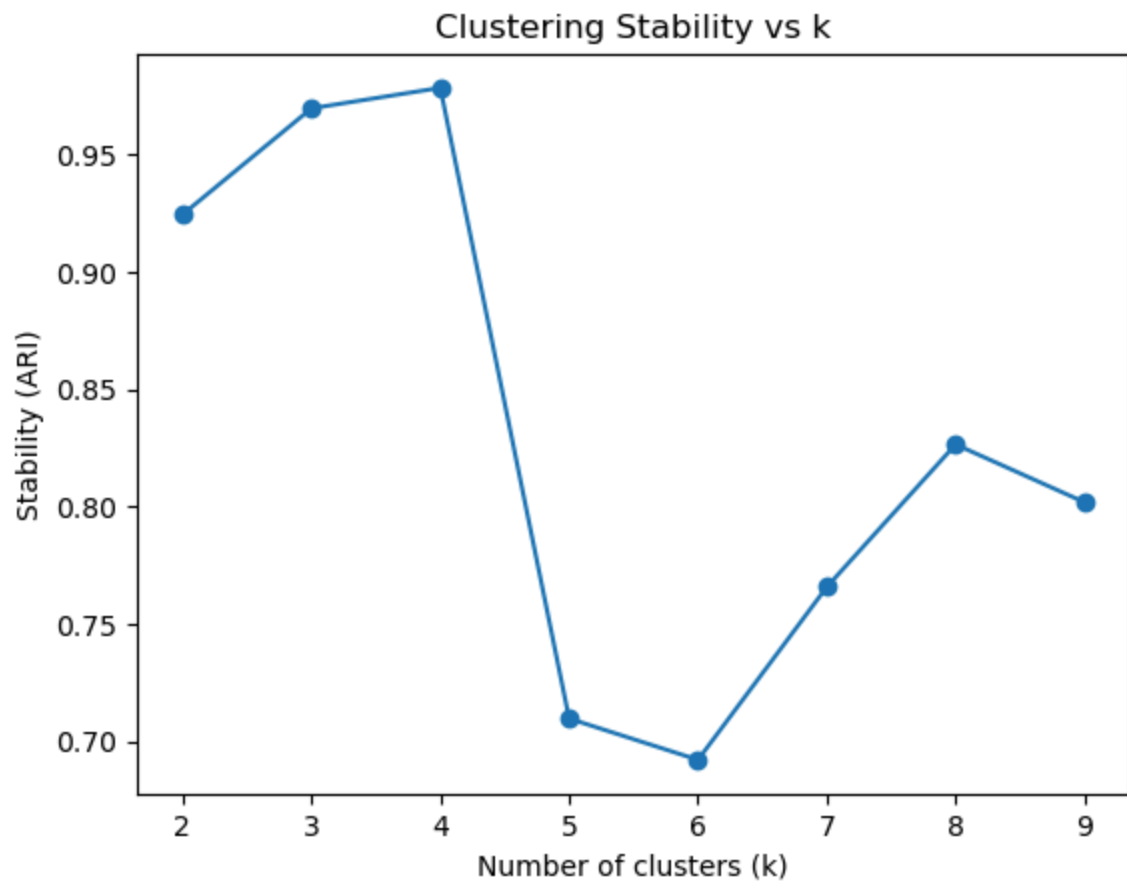
```

```

In [14]: # Plot Results

plt.plot(k_values, stability_scores, marker='o')
plt.xlabel("Number of clusters (k)")
plt.ylabel("Stability (ARI)")
plt.title("Clustering Stability vs k")
plt.show()

```



4. Final K-Means Model & Visualization

We now run the model with K=4 and visualize the results and their centers (centroids).

```
In [15]: final_kmeans = KMeans(n_clusters=4, random_state=0, n_init=10)
df['clusters'] = final_kmeans.fit_predict(X)
```

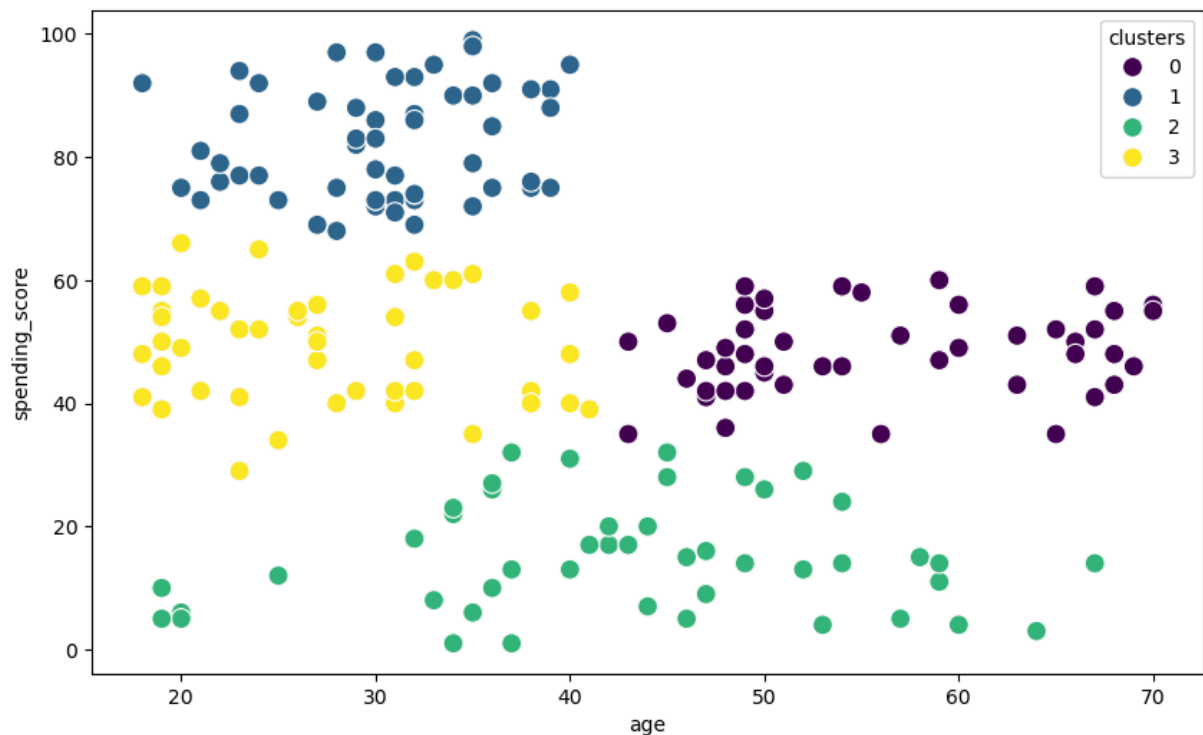
```
In [16]: df
```

Out[16]:

	age	spending_score	clusters
0	19	39	3
1	21	81	1
2	20	6	2
3	23	77	1
4	31	40	3
...	...	...	...
195	35	79	1
196	45	28	2
197	32	74	1
198	32	18	2
199	30	83	1

200 rows x 3 columns

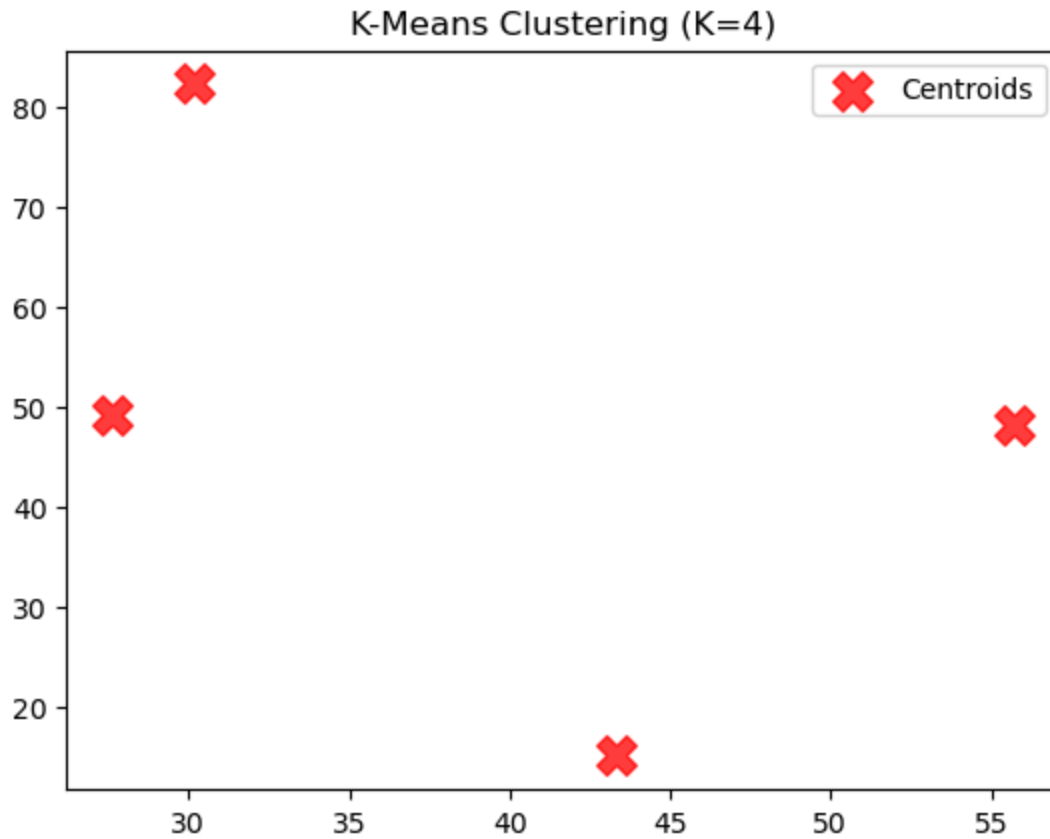
```
In [17]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='age', y='spending_score', hue='clusters', palette='magma')
```



```
In [18]: # Plotting Centers
centers = final_kmeans.cluster_centers_
centers
```

```
Out[18]: array([[55.70833333, 48.22916667],  
               [30.1754386 , 82.35087719],  
               [43.29166667, 15.02083333],  
               [27.61702128, 49.14893617]])
```

```
In [19]: plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='x')  
plt.title("K-Means Clustering (K=4)")  
plt.legend()  
plt.show()
```



```
In [23]: df
```

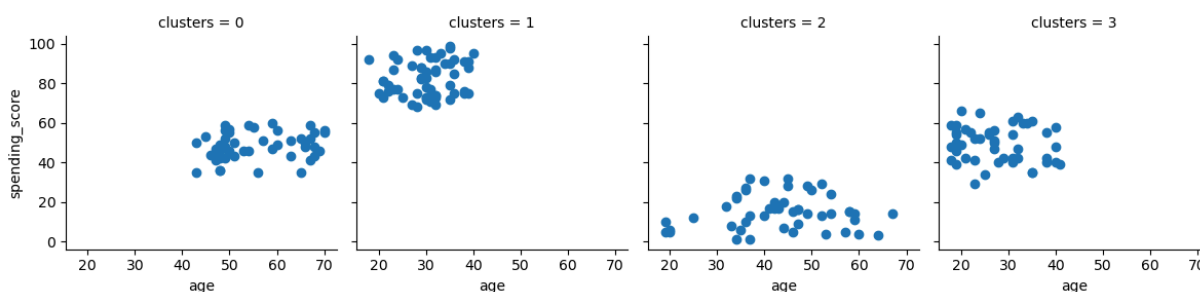
Out[23]:

	age	spending_score	clusters
0	19	39	3
1	21	81	1
2	20	6	2
3	23	77	1
4	31	40	3
...	...	...	...
195	35	79	1
196	45	28	2
197	32	74	1
198	32	18	2
199	30	83	1

	age	spending_score	clusters
0	19	39	3
1	21	81	1
2	20	6	2
3	23	77	1
4	31	40	3
...	...	...	...
195	35	79	1
196	45	28	2
197	32	74	1
198	32	18	2
199	30	83	1

200 rows × 3 columns

In [24]: `sns.FacetGrid(data=df, col='clusters').map(plt.scatter, 'age', 'spending_score')`



Business Conclusions

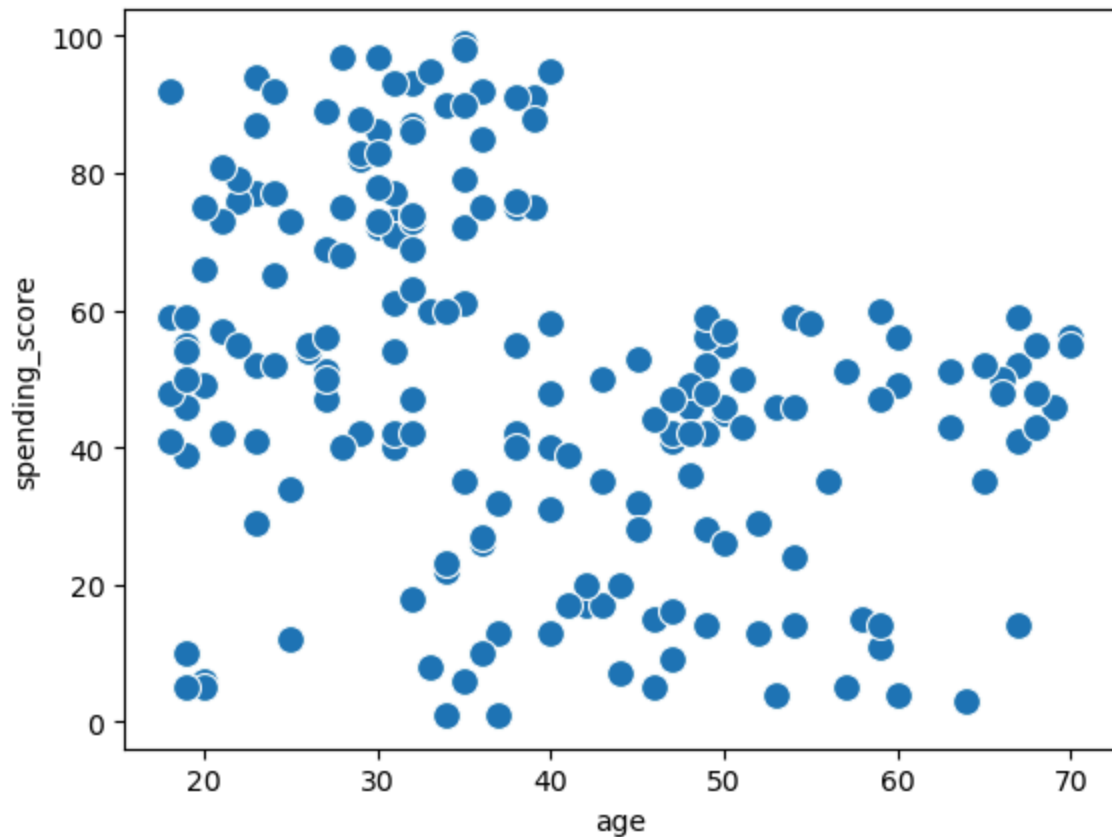
By segmenting the data, we can identify four distinct customer personas:

1. **Cluster 0:** Middle-age customers with low spending.
2. **Cluster 1:** Young customers with high spending.
3. **Cluster 2:** Young customers with moderate spending.
4. **Cluster 3:** Senior customers with moderate spending.

5. Hierarchical Clustering & Dendrograms

Hierarchical clustering builds a tree of clusters. We use **Ward's Linkage**, which minimizes the variance (ESS) when merging clusters.

In [25]: `sns.scatterplot(data=df, x='age', y='spending_score', s=100);`



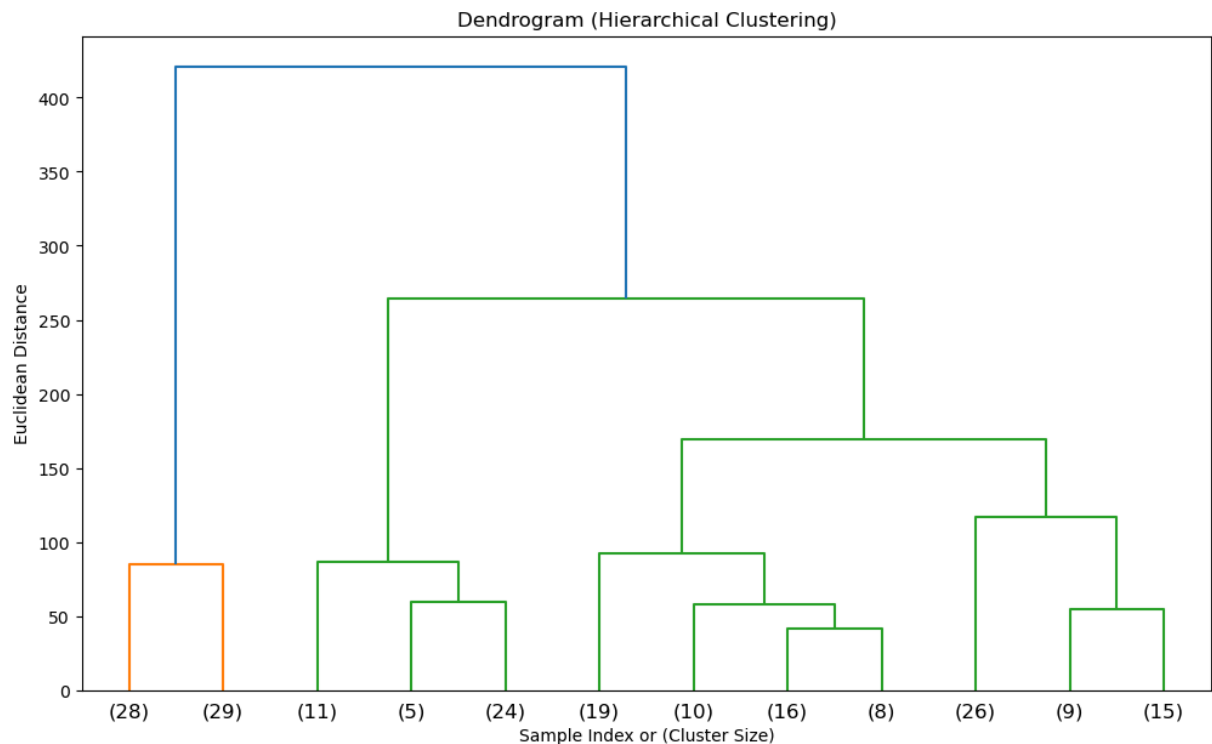
```
In [26]: # Agglomerative Clustering (Bottom-Up)
from sklearn.cluster import AgglomerativeClustering
my_agg= AgglomerativeClustering(4)
my_agg.fit(df)
```

```
Out[26]: ▼ AgglomerativeClustering ⓘ ?
          ► Parameters
```

```
In [ ]:
```

```
In [27]: # Create Linkage Matrix
from scipy.cluster.hierarchy import linkage, dendrogram
my_lnk = linkage(X, method='ward')

plt.figure(figsize=(12, 7))
dendrogram(my_lnk, orientation='top', truncate_mode='lastp', p=12)
plt.title('Dendrogram (Hierarchical Clustering)')
plt.xlabel('Sample Index or (Cluster Size)')
plt.ylabel('Euclidean Distance')
plt.show()
```



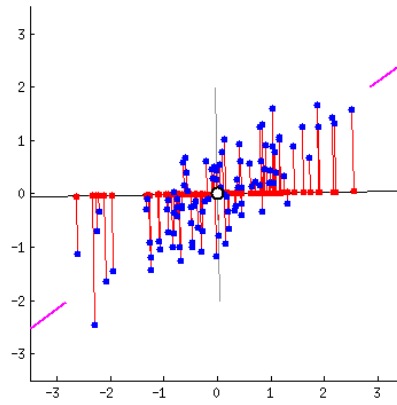
6. Dimensionality Reduction

6.1 PCA

- Principal Component Analysis (PCA) is a technique used to reduce the number of features in a dataset while preserving as much information (variance) as possible.
- Instead of using the original variables directly, PCA creates new variables called principal components:
 - 1st principal component = direction with the most variance
 - 2nd principal component = next-most variance, orthogonal to the first
 - 3rd principal component = next-most variance, orthogonal to the first two
 - and so on

Typical goals of PCA:

- Reduce dimensionality
- Remove redundant / correlated features
- Speed up models
- Visualize high-dimensional data in 2D or 3D
- Reduce noise



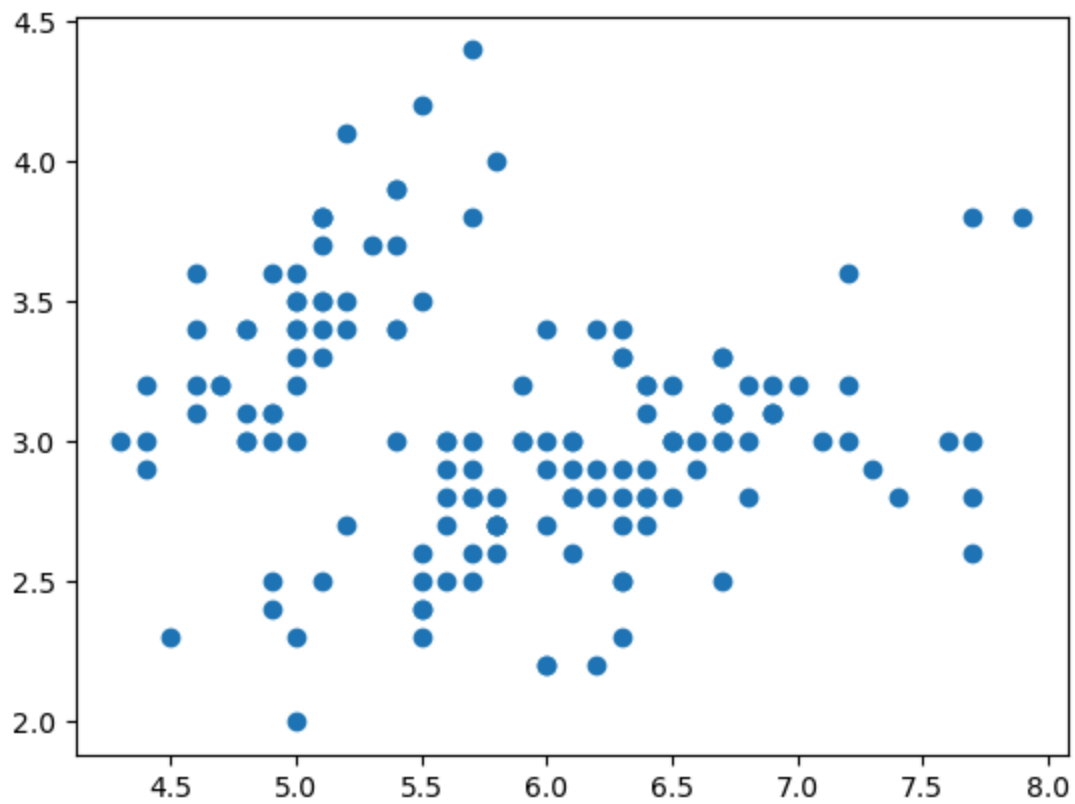
```
In [30]: # Example using iris data
from sklearn.datasets import load_iris
data=load_iris().data
data
```

```
Out[30]: array([[5.1, 3.5, 1.4, 0.2],
 [4.9, 3. , 1.4, 0.2],
 [4.7, 3.2, 1.3, 0.2],
 [4.6, 3.1, 1.5, 0.2],
 [5. , 3.6, 1.4, 0.2],
 [5.4, 3.9, 1.7, 0.4],
 [4.6, 3.4, 1.4, 0.3],
 [5. , 3.4, 1.5, 0.2],
 [4.4, 2.9, 1.4, 0.2],
 [4.9, 3.1, 1.5, 0.1],
 [5.4, 3.7, 1.5, 0.2],
 [4.8, 3.4, 1.6, 0.2],
 [4.8, 3. , 1.4, 0.1],
 [4.3, 3. , 1.1, 0.1],
 [5.8, 4. , 1.2, 0.2],
 [5.7, 4.4, 1.5, 0.4],
 [5.4, 3.9, 1.3, 0.4],
 [5.1, 3.5, 1.4, 0.3],
 [5.7, 3.8, 1.7, 0.3],
 [5.1, 3.8, 1.5, 0.3],
 [5.4, 3.4, 1.7, 0.2],
 [5.1, 3.7, 1.5, 0.4],
 [4.6, 3.6, 1. , 0.2],
 [5.1, 3.3, 1.7, 0.5],
 [4.8, 3.4, 1.9, 0.2],
 [5. , 3. , 1.6, 0.2],
 [5. , 3.4, 1.6, 0.4],
 [5.2, 3.5, 1.5, 0.2],
 [5.2, 3.4, 1.4, 0.2],
 [4.7, 3.2, 1.6, 0.2],
 [4.8, 3.1, 1.6, 0.2],
 [5.4, 3.4, 1.5, 0.4],
 [5.2, 4.1, 1.5, 0.1],
 [5.5, 4.2, 1.4, 0.2],
 [4.9, 3.1, 1.5, 0.2],
 [5. , 3.2, 1.2, 0.2],
 [5.5, 3.5, 1.3, 0.2],
 [4.9, 3.6, 1.4, 0.1],
 [4.4, 3. , 1.3, 0.2],
 [5.1, 3.4, 1.5, 0.2],
 [5. , 3.5, 1.3, 0.3],
 [4.5, 2.3, 1.3, 0.3],
 [4.4, 3.2, 1.3, 0.2],
 [5. , 3.5, 1.6, 0.6],
 [5.1, 3.8, 1.9, 0.4],
 [4.8, 3. , 1.4, 0.3],
 [5.1, 3.8, 1.6, 0.2],
 [4.6, 3.2, 1.4, 0.2],
 [5.3, 3.7, 1.5, 0.2],
 [5. , 3.3, 1.4, 0.2],
 [7. , 3.2, 4.7, 1.4],
 [6.4, 3.2, 4.5, 1.5],
 [6.9, 3.1, 4.9, 1.5],
 [5.5, 2.3, 4. , 1.3],
 [6.5, 2.8, 4.6, 1.5],
 [5.7, 2.8, 4.5, 1.3],
```

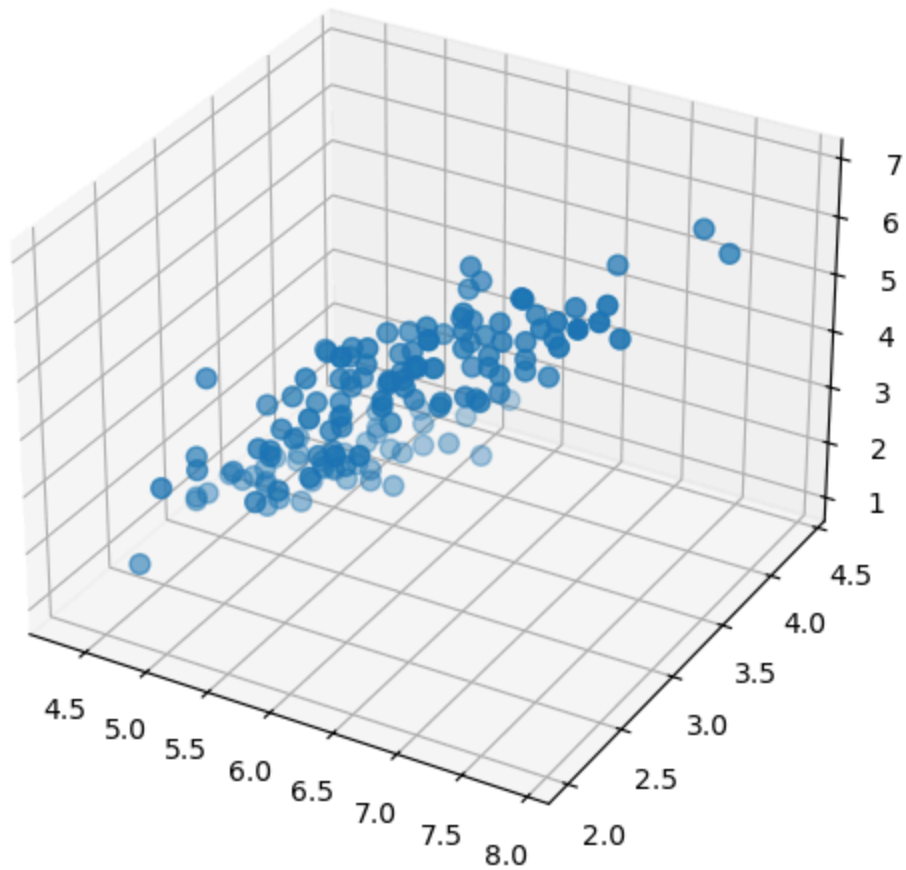
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1.],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1.],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1.],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1.],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1.],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1.],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1.],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2.],
[6.4, 2.7, 5.3, 1.9],

```
[6.8, 3. , 5.5, 2.1],  
[5.7, 2.5, 5. , 2. ],  
[5.8, 2.8, 5.1, 2.4],  
[6.4, 3.2, 5.3, 2.3],  
[6.5, 3. , 5.5, 1.8],  
[7.7, 3.8, 6.7, 2.2],  
[7.7, 2.6, 6.9, 2.3],  
[6. , 2.2, 5. , 1.5],  
[6.9, 3.2, 5.7, 2.3],  
[5.6, 2.8, 4.9, 2. ],  
[7.7, 2.8, 6.7, 2. ],  
[6.3, 2.7, 4.9, 1.8],  
[6.7, 3.3, 5.7, 2.1],  
[7.2, 3.2, 6. , 1.8],  
[6.2, 2.8, 4.8, 1.8],  
[6.1, 3. , 4.9, 1.8],  
[6.4, 2.8, 5.6, 2.1],  
[7.2, 3. , 5.8, 1.6],  
[7.4, 2.8, 6.1, 1.9],  
[7.9, 3.8, 6.4, 2. ],  
[6.4, 2.8, 5.6, 2.2],  
[6.3, 2.8, 5.1, 1.5],  
[6.1, 2.6, 5.6, 1.4],  
[7.7, 3. , 6.1, 2.3],  
[6.3, 3.4, 5.6, 2.4],  
[6.4, 3.1, 5.5, 1.8],  
[6. , 3. , 4.8, 1.8],  
[6.9, 3.1, 5.4, 2.1],  
[6.7, 3.1, 5.6, 2.4],  
[6.9, 3.1, 5.1, 2.3],  
[5.8, 2.7, 5.1, 1.9],  
[6.8, 3.2, 5.9, 2.3],  
[6.7, 3.3, 5.7, 2.5],  
[6.7, 3. , 5.2, 2.3],  
[6.3, 2.5, 5. , 1.9],  
[6.5, 3. , 5.2, 2. ],  
[6.2, 3.4, 5.4, 2.3],  
[5.9, 3. , 5.1, 1.8]])
```

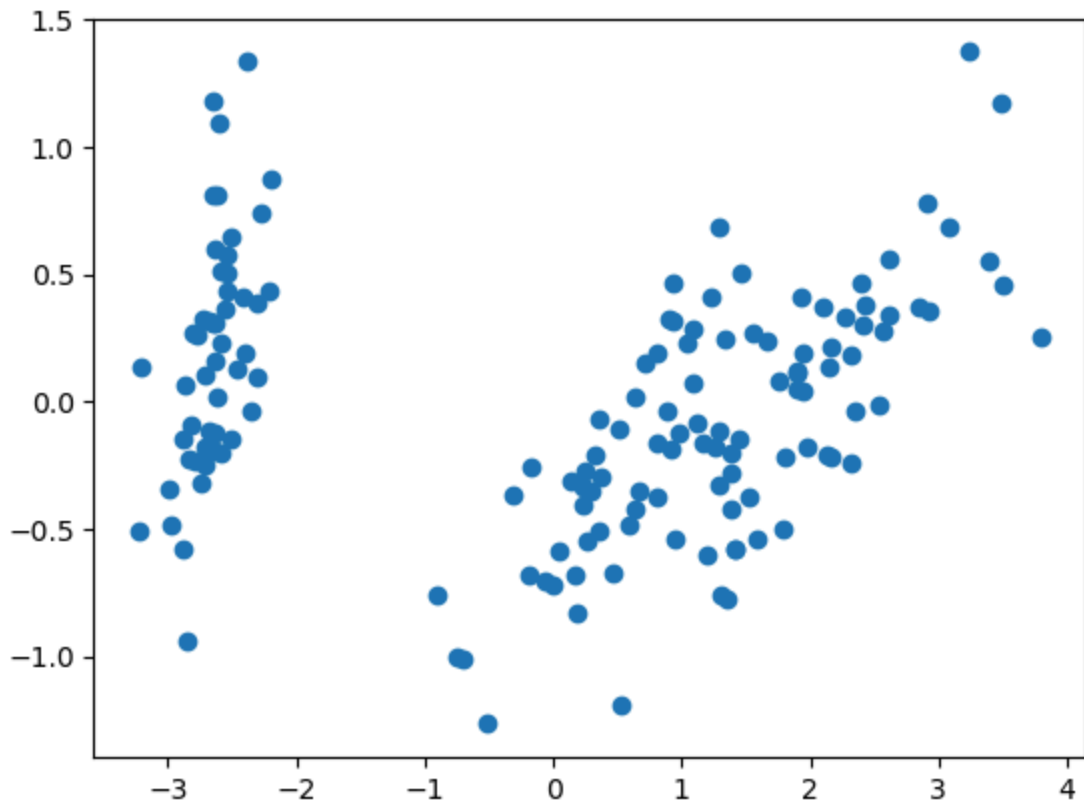
```
In [31]: plt.scatter(data[:,0],data[:,1]);
```



```
In [32]: fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter3D(data[:,0], data[:,1], data[:,2], s=50);
```

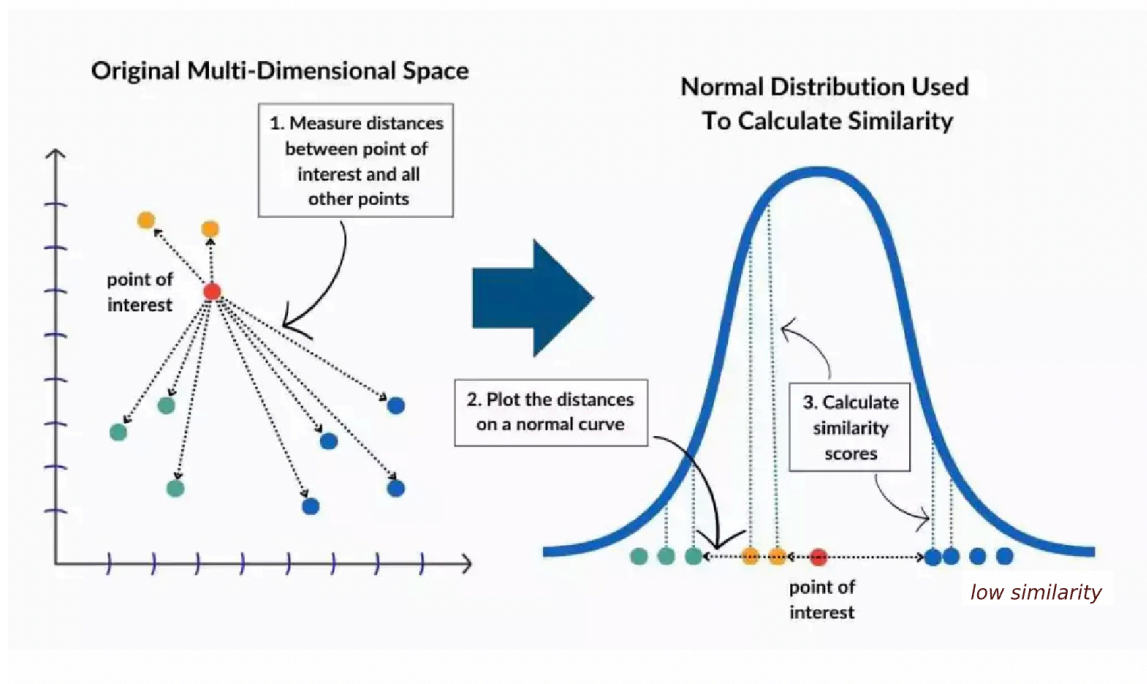


```
In [33]: # PCA Example
from sklearn.decomposition import PCA
my_pca= PCA(n_components=2)
new_data=my_pca.fit_transform(data)
plt.scatter(new_data[:,0],new_data[:,1]);
```



6.2 t-SNE

- t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear dimensionality reduction technique mainly used for visualization of high-dimensional data in 2D or 3D.
- Unlike PCA, which tries to preserve overall variance, t-SNE focuses on preserving the neighborhood around each point:
 - Points that are close in the original space stay close
 - Distant points may not preserve their exact distances



For each point (x_i):

1. Treat (x_i) as the current point.
2. Compute how close every other point (x_j) is to it
3. Create neighbor probabilities ($p_{j|i}$)
4. Try to place that point in low-dimensional space so its nearby neighbors stay nearby

For example, with 100 samples:

- Point 1 asks: “Which points are closest to me?”
- Point 2 asks the same
- ...
- Point 100 asks the same

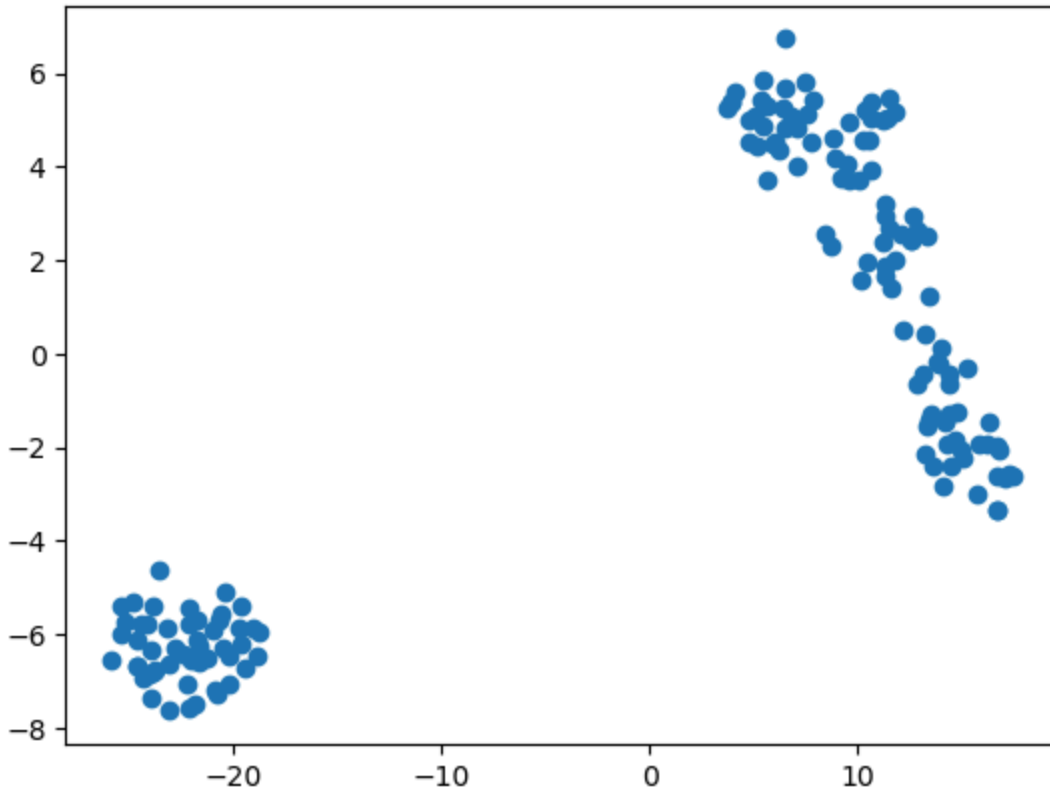
Then t-SNE finds a 2D layout that best satisfies all of those neighborhood relationships simultaneously.

This makes t-SNE very good for finding clusters or groups visually.

PCA	t-SNE
Linear	Nonlinear
Preserves variance	Preserves local similarity
Fast	Slower
Good for preprocessing	Good for visualization
Components are interpretable	Axes are not interpretable

In practice, many people do: `Data → PCA → t-SNE` because PCA removes noise and makes t-SNE faster.

```
In [34]: # TSNE Example
from sklearn.manifold import TSNE
my_tsne = TSNE(n_components=2, init='pca')
new_data=my_tsne.fit_transform(data)
plt.scatter(new_data[:,0],new_data[:,1]);
```



```
In [ ]:
```